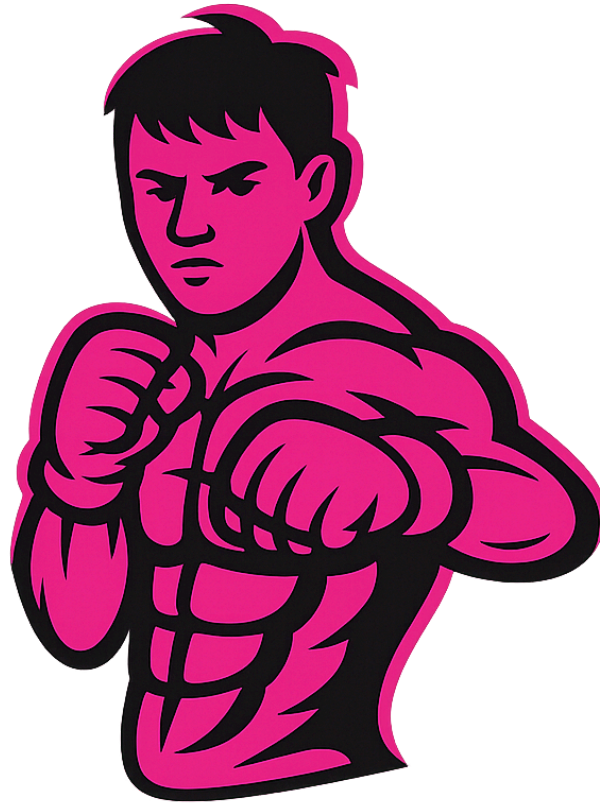




FIGHTCLUBX

Alumno: Pablo Méndez García
Tutor: Abel Morillo Sánchez
Centro: Ceu San Pablo Andalucía
Ciclo:D.A.M
Fecha: 21/01/2026



CEU

*Centro de Estudios
Profesionales*

Fundación San Pablo Andalucía

Índice

1. Introducción y objetivos	3
1.1. Introducción del proyecto	3
1.2. Objetivos del proyecto	3
2. Contexto del proyecto	4
2.1. Estado actual	4
2.2. Justificación del proyecto	5
3. Análisis de requisitos	14
4. Planificación	15
4.1. Planificación	15
4.1.1 Preparación (Enero 2026)	15
4.1.2 Diseño UX/UI (Febrero 2026)	16
4.1.3 Desarrollo MVP (Marzo – Abril 2026)	16
4.1.4 Pruebas con usuarios (Abril – Mayo 2026)	16
4.1.3.1 Configuración del entorno de desarrollo	16
4.1.3.2 Desarrollo del sistema de autenticación y usuarios	17
4.1.3.3 Desarrollo de la gestión de ligas	17
4.1.3.4 Desarrollo del sistema de combates	17
4.1.3.5 Desarrollo del sistema de torneos	18
4.1.3.6 Desarrollo del sistema de ranking	18
4.1.3.7 Desarrollo del sistema de notificaciones	18
4.1.3.8 Integración de publicidad	19
4.1.3.9 Integración con la base de datos	19
4.1.3.10 Revisión y ajustes del MVP	19
4.1.5 Lanzamiento (Mayo 2026)	20
4.2. Metodología	20
5. Diseño de solución	21
5.1. Tecnologías utilizadas	21
5.2. Objetivos esperados del diseño	25
6. Ejecución	27
7. Pruebas y despliegue	31
8. Conclusiones y propuestas de mejoras	33
8.1. Conclusiones	33
8.2. Propuestas de mejora	33
9. Bibliografía	33

1. Introducción y objetivos

1.1. Introducción del proyecto

Este proyecto consiste en una app para dispositivos móviles llamada “FightClubX” ,la cual proporcionará a los usuarios diversos sistemas para organizar eventos de deporte de combate de una manera distendida y cómoda.

Esta app se inspira en el fenómeno social de las ligas fantásticas deportivas, la aplicación da una alternativa para las personas aficionadas a los deportes de contacto, y añade el factor de los eventos reales a las ligas y eventos.

A su vez la app fomenta el sentimiento de camaradería y los valores que transmiten los deportes ,realizando así el crecimiento personal que se suele adquirir al practicar dichas disciplinas

1.2. Objetivos del proyecto

A la hora del desarrollo los objetivos a conseguir serían los siguientes:

- **Centralización y simplificación de eventos:**

El proyecto busca centralizar la gestión de los eventos casuales y amateurs de deportes de contacto organizados de una manera simple y cómoda otorgando facilidades a los usuarios en el nivel de gestión

- **Otorgar alternativas a los usuarios casuales:**

Existen sistemas de gestión para estos eventos ,pero solo son enfocados para eventos deportivos a nivel competitivo y solo tienen acceso las federaciones deportivas.

La aplicación brinda una alternativa para aquellos usuarios que desean gestionar sus encuentros deportivos a un nivel más casual ,por ejemplo un grupo de amigos que practican algún tipo de deporte de combate ,actualmente no tiene ninguna alternativa para organizar sus eventos amistosos.

- **Remuneración económica:**

A nivel económico la app busca realizar ingresos a través de un sistema de publicidad pasiva el cual habrá banners publicitarios en la parte inferior de la pantalla para que las marcas interesadas se publiciten en ellos.

2. Contexto del proyecto

2.1. Estado actual

A día de hoy, la organización de ligas, torneos y combates para personas que practican deportes de combate de manera amateur, o como afición, suele verse limitada por la falta de herramientas digitales dedicadas específicamente a ello. Los eventos suelen depender de mensajerías, hojas de cálculo y organización manual, generando problemas en los emparejamientos, seguimiento de resultados y registro de participantes. Es ahí donde surge FightClubX para solucionar esta serie de problemas proponiendo una app que centraliza la gestión de perfiles, ligas ,rankings, torneos y combates.

En ella se podrán organizar todo tipo de combates,torneos y ligas de artes marciales, sin importar el deporte que practiques .Ha diferencia de otras apps que se centran en pequeños grupos de deportes por ejemplo englobando disciplinas de grappling como Scorizer, en FightClubX permitirá gestionar todo tipo de eventos con posibilidades de adaptar los rounds ,tiempos y ligas a las necesidades de los usuarios.

Por último un factor clave es la accesibilidad a los usuarios simplemente practicantes de estos deportes, para abarcar un público que ha día de hoy no cuenta con opciones para gestionar los eventos mencionados.

2.2. Justificación del proyecto

El principal motivo de la creación de FightClubX fue una necesidad personal debido a la falta de opciones en el sistema de gestión de eventos casuales de Artes Marciales. A raíz de su planteamiento surgieron otros motivos explicados a continuación :

-La remuneración económica a través de un sistema de monetización pasiva a través de banners publicitarios en la app.

-La posibilidad de innovar en el mercado ,ya que a día de hoy no existen aplicaciones que cumplan los requisitos que FightClubX cubre.

-El auge a nivel mundial de los deportes de combate

Varios factores impulsan el auge comentado¹:

- ☐ 1. Mayor presencia en los medios: las principales promociones, incluidas UFC, ONE Championship y PFL, han conseguido acuerdos a gran escala con plataformas de televisión y transmisión, lo que ha aumentado significativamente su audiencia.
- ☐ 2. Globalización: Organizar eventos en nuevas regiones y países, atrayendo a luchadores de todo el mundo.
- ☐ 3. Digitalización: Uso activo de redes sociales y plataformas digitales para promocionar luchadores y eventos.

¹Fuente: [PTF-lab](#)

- ☐ 4. Personalidades estelares: El surgimiento de luchadores carismáticos con grandes bases de seguidores, como Conor McGregor, Khabib Nurmagomedov e Israel Adesanya.

3. Análisis de requisitos

FigthClubX plantea registrar usuarios, los cuales contarán con herramientas para gestionar sus eventos, siendo estos de tres tipos distintos: combate, torneo, liga. Todas estas funciones se desarrollaran por completo a continuación junto a su etiqueta MoSCoW

Requisitos Funcionales

1. Gestión de usuarios y roles:

1.1(**M**): El sistema debe permitir el registro de usuarios con datos personales (email, contraseña, nombre, edad, peso, altura, disciplina, años de práctica).

1.2(**M**): El sistema debe asignar roles dentro de una liga: Presidente, Vicepresidente, Luchador, Árbitro o Espectador

1.3(**S**): El Presidente podrá modificar o retirar permisos a otros usuarios, expulsar miembros y asignar cargos.

2. Gestión de ligas

2.1(**M**): Los usuarios con permisos (Presidente/Vicepresidente) podrán crear ligas personalizadas con reglas propias (número de rounds, duración, tipo de combate, normas específicas).

2.2(**M**): Los usuarios podrán unirse a una liga existente o abandonarla.

2.3(**M**): El presidente podrá expulsar o cambiar el rol del resto de integrantes de la liga

2.4(**S**): Cada liga contará con un sistema de ranking dinámico basado en resultados y datos del perfil de cada luchador.

3. Torneos

3.1(**M**): El sistema debe permitir crear torneos con un mínimo de tres participantes (dos luchadores y un árbitro).

3.2(**M**): El sistema debe generar automáticamente los emparejamientos de los combates.

3.3(**S**): El sistema debe permitir visualizar los cuadros de torneo y el avance de los combatientes.

4. Combates rápidos

4.1(**M**): El sistema debe permitir crear combates individuales entre tres participantes (dos luchadores y un árbitro).

4.2(**M**): Debe incluir un **cronómetro integrado**, contador de rounds y temporizador configurable.

4.3(**M**): El árbitro podrá pausar o finalizar el combate desde su panel de control.

4.4(**S**): Los resultados del combate se registran automáticamente en el historial de los luchadores y, si pertenecen a una liga, afectarán al ranking.

5. Encuestas y votaciones

5.1(**C**): El Presidente y Vicepresidente podrán crear encuestas (por ejemplo, para reglas o decisiones arbitrales).

5.2(C): Los espectadores podrán votar en tiempo real cuando el árbitro solicite una decisión dudosa.

6. Sistema de ranking

6.1(S): Cada liga tendrá un ranking que combine resultados de combates con atributos de perfil (peso, experiencia, victorias, derrotas).

6.2(S): Los cambios en el ranking se reflejarán automáticamente tras cada combate o torneo.

7. Notificaciones

7.1(S): El sistema debe notificar a través de las notificaciones de teléfono a los usuarios sobre próximos torneos, combates y votaciones abiertas.

7.2(C): Los usuarios podrán personalizar sus preferencias de notificación.

8. Historial de combates

8.1(S): Cada usuario podrá consultar su historial de combates y torneos con resultados, fechas y estadísticas.

8.2(C): Los árbitros podrán añadir notas o incidencias tras cada combate.

Requisitos No Funcionales

1. Rendimiento

1.1(M): La app debe cargar en menos de 3 segundos

1.2(S): La app debe permitir un uso fluido en torneos con más de 100 usuarios activos.

2. Seguridad

2.1(**M**): Los datos personales se cifran tanto en tránsito como en reposo.

2.2(**S**): Se implementará autenticación de dos factores para usuarios con roles de Presidente o Vicepresidente.

3. Disponibilidad

F3.1(**S**): La aplicación debe tener una disponibilidad mínima del 99,5%.

4. Escalabilidad

4.1(**M**): La arquitectura permitirá añadir nuevas funcionalidades (votaciones en vivo, estadísticas) sin rediseño completo.

5. Usabilidad

5.1(**M**): La interfaz debe ser intuitiva, responsive y accesible en móvil y tablet

6. Interoperabilidad

6.1(**C**): El sistema deberá integrarse con servicios de autenticación social (Google/Facebook) y con APIs de calendario y notificaciones push.

7. Mantenibilidad

7.1(**M**): El código seguirá estándares de programación y documentación modular para facilitar futuras mejoras.

Requisitos de datos

El sistema deberá almacenar la siguiente información para su correcto funcionamiento:

☐ **Datos de usuario**

El sistema deberá almacenar la información necesaria de los usuarios registrados.

Datos requeridos:

- Identificador único del usuario
- Nombre
- Correo electrónico
- Contraseña
- Edad
- Peso
- Altura
- Fecha de registro

Restricciones:

- El correo electrónico debe ser único.
- La contraseña deberá almacenarse de forma segura.
- Un usuario podrá estar asociado a una o varias ligas.
- Un usuario podrá practicar una o varias disciplinas.

☐ **Datos de disciplinas**

El sistema deberá gestionar las distintas disciplinas deportivas disponibles.

Datos requeridos:

- Identificador de disciplina
- Nombre de la disciplina
- Descripción

Restricciones:

- Una disciplina puede estar asociada a múltiples usuarios.
- Un usuario puede practicar varias disciplinas.

☐ **Datos de ligas**

Las ligas permiten agrupar usuarios y organizar combates y torneos.

Datos requeridos:

- Identificador de la liga
- Nombre
- Descripción
- Reglas
- Fecha de creación
- Estado
- Usuario creador

Restricciones:

- Cada liga deberá tener un usuario creador.
- Una liga podrá tener múltiples usuarios asociados.
- Una liga podrá contener combates y torneos.

☐ **Datos de roles**

Los roles definen los permisos de los usuarios dentro de una liga.

Datos requeridos:

- Identificador del rol
- Nombre del rol

Restricciones:

- Un mismo rol podrá asignarse a varios usuarios.

☐ **Datos de combates**

El sistema deberá almacenar la información de los combates organizados.

Datos requeridos:

- Identificador del combate
- Tipo de combate
- Número de rounds
- Duración de los rounds
- Estado
- Fecha del combate
- Liga asociada (opcional)

Restricciones:

- Un combate deberá tener al menos dos participantes.
- Un combate podrá estar asociado a una liga o ser independiente.

☐ **Datos de participantes en combates**

Se deberá registrar la participación de los usuarios en cada combate.

Datos requeridos:

- Identificador del participante
- Usuario participante
- Combate asociado
- Rol dentro del combate

Restricciones:

- Un usuario no podrá participar más de una vez en el mismo combate.
- Un combate podrá tener varios participantes con distintos roles.

☐ **Datos de resultados de combates**

El sistema deberá almacenar el resultado final de cada combate.

Datos requeridos:

- Identificador del resultado
- Combate asociado
- Ganador
- Método de victoria
- Observaciones

Restricciones:

- Cada combate tendrá un único resultado.
- El ganador deberá ser uno de los participantes del combate.

☐ **Datos de torneos**

Los torneos agrupan varios combates dentro de una liga.

Datos requeridos:

- Identificador del torneo
- Nombre
- Formato
- Estado
- Fecha de inicio
- Fecha de fin
- Liga asociada

Restricciones:

- Un torneo deberá pertenecer a una única liga.
- Un torneo podrá contener múltiples combates.

☐ **Datos de ranking**

El sistema deberá mantener una clasificación de los usuarios dentro de cada liga.

Datos requeridos:

- Identificador del ranking
- Liga asociada
- Usuario
- Número de victorias
- Número de derrotas
- Posición

Restricciones:

- Un usuario solo podrá aparecer una vez en el ranking de una liga.
- El ranking se actualizará tras cada combate.

☐ **Datos de notificaciones**

El sistema deberá gestionar las notificaciones enviadas a los usuarios.

Datos requeridos:

- Usuario destinatario
- Mensaje

Restricciones:

- Un usuario podrá recibir múltiples notificaciones.

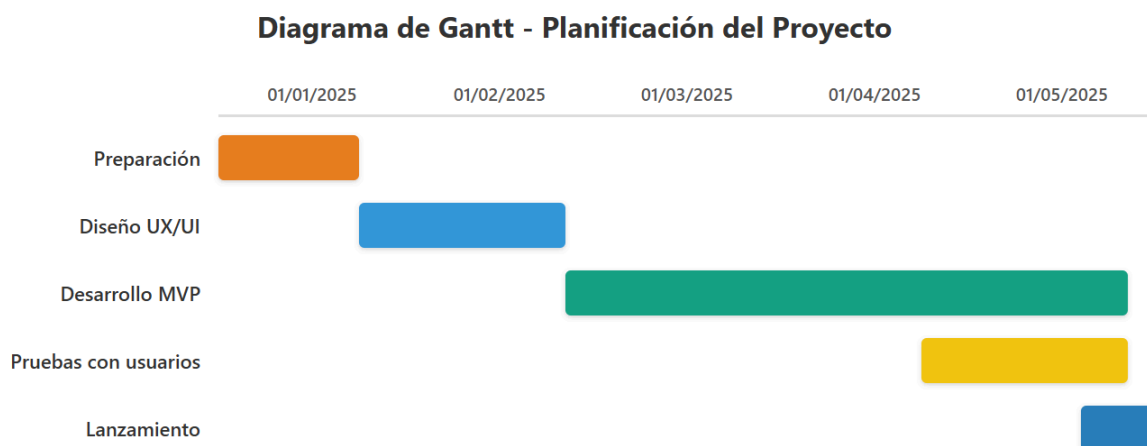
4. Planificación

4.1. Planificación

La planificación del proyecto se ha realizado mediante la metodología Gantt, que permite organizar las distintas fases del proyecto en el tiempo, establecer su duración y visualizar la secuencia de ejecución. Esta metodología facilita el control del avance y la correcta coordinación entre las tareas.

Para poder planificar mejor el proyecto se utilizara Jira como herramienta de gestión, para así poder planificar con mayor exactitud cada uno de los puntos.

El proyecto se desarrolla entre los meses de enero y mayo de 2026, y se divide en cinco fases principales, tal y como se representa en el diagrama de Gantt a continuación.



4.1.1 Preparación (Enero 2026)

En esta fase inicial se definen las bases del proyecto. Se realiza un análisis general de la idea, se establecen los objetivos, el alcance y los requisitos principales. También se recopila la información necesaria y se planifican los recursos que se utilizarán durante el desarrollo.

Esta etapa es fundamental para asegurar que el proyecto tenga una dirección clara antes de comenzar con el diseño y desarrollo.

4.1.2 Diseño UX/UI (Febrero 2026)

Durante esta fase se diseñará la experiencia de usuario (UX) y la interfaz gráfica (UI) del proyecto. Se crearán bocetos y prototipos que definan la estructura visual y la navegación de la aplicación.

El objetivo es garantizar una interfaz intuitiva, atractiva y fácil de usar, sirviendo como guía para la fase de desarrollo posterior.

4.1.3 Desarrollo MVP (Marzo – Abril 2026)

En esta fase se llevará a cabo el desarrollo del MVP (Producto Mínimo Viable). Se implementan las funcionalidades principales del proyecto siguiendo el diseño previamente definido.

El MVP permite disponer de una versión funcional de la app, centrada en las características esenciales, que servirá para realizar pruebas reales y detectar posibles mejoras antes del lanzamiento final.

4.1.4 Pruebas con usuarios (Abril – Mayo 2026)

Una vez desarrollado el MVP, se realizan pruebas con usuarios para evaluar el funcionamiento de la app y la experiencia de uso. Se recopilan opiniones, se detectan errores y se analizan posibles mejoras.

El desarrollo del MVP se divide en las siguientes subtareas:

4.1.3.1 Configuración del entorno de desarrollo

En esta subtarea se realizará la preparación del entorno de trabajo necesario para el desarrollo de la aplicación.

Incluye:

- Configuración del entorno de desarrollo con Ionic, React y TypeScript.
- Configuración del repositorio de control de versiones mediante Git.
- Creación de la estructura inicial del proyecto.
- Configuración de herramientas de desarrollo y depuración.

4.1.3.2 Desarrollo del sistema de autenticación y usuarios

Se implementará la gestión básica de usuarios de la aplicación.

Incluye:

- Registro de usuarios.
- Inicio de sesión.
- Gestión de perfiles de usuario.
- Almacenamiento seguro de credenciales.

4.1.3.3 Desarrollo de la gestión de ligas

En esta subtarea se desarrollarán las funcionalidades relacionadas con la creación y administración de ligas.

Incluye:

- Creación de ligas personalizadas.
- Unión y salida de usuarios de una liga.
- Asignación de roles dentro de la liga.
- Gestión de permisos (Presidente, Vicepresidente, Luchador, Árbitro, Espectador).

4.1.3.4 Desarrollo del sistema de combates

Se implementará la funcionalidad de creación y gestión de combates.

Incluye:

- Creación de combates rápidos.
- Asignación de participantes (luchadores y árbitro).
- Configuración de rounds y tiempos.
- Implementación del cronómetro y control del combate.
- Finalización y registro del combate.

4.1.3.5 Desarrollo del sistema de torneos

Se desarrollará el módulo de torneos ,incluyendo los torneos en ligas.

Incluye:

- Creación de torneos.
- Generación automática de emparejamientos.
- Visualización del progreso del torneo.
- Asociación de combates a un torneo.

4.1.3.6 Desarrollo del sistema de ranking

Se implementará el sistema de clasificación de usuarios dentro de las ligas.

Incluye:

- Cálculo de victorias y derrotas.
- Actualización automática del ranking tras cada combate.
- Visualización del ranking por liga.

4.1.3.7 Desarrollo del sistema de notificaciones

Se desarrollará el sistema de notificaciones para informar a los usuarios de eventos relevantes.

Incluye:

- Envío de notificaciones sobre combates y torneos.
- Avisos de cambios en ligas.
- Gestión básica de notificaciones dentro de la app.

4.1.3.8 Integración de publicidad

Se realizará la integración del sistema de monetización mediante publicidad.

Incluye:

- Integración de Google AdMob.
- Implementación de banners publicitarios.
- Ajuste de la visualización para no afectar negativamente a la experiencia de usuario.

4.1.3.9 Integración con la base de datos

Se implementará la conexión entre la aplicación y la base de datos.

Incluye:

- Creación de las tablas necesarias en MariaDB.
- Implementación de operaciones CRUD.
- Validación de datos y control de errores.
- Comprobación de la integridad de la información almacenada.

4.1.3.10 Revisión y ajustes del MVP

Finalmente, se realizará una revisión general del MVP desarrollado.

Incluye:

- Corrección de errores detectados.
- Ajustes de rendimiento.
- Revisión de la usabilidad.
- Preparación del MVP para la fase de pruebas con usuarios.

4.1.5 Lanzamiento (Mayo 2026)

En la última fase se realiza el lanzamiento del proyecto. Se aplican los últimos ajustes derivados de las pruebas, se prepara la versión final y se entrega el producto terminado.

Esta etapa marca el cierre del proyecto, asegurando que el sistema está listo para su uso y cumple con los objetivos planteados inicialmente.

4.2. Metodología

Para el desarrollo del proyecto se utilizará la metodología ágil Scrum, adaptada a un proyecto individual. Esta metodología permite organizar el trabajo de forma iterativa e incremental, facilitando el control del progreso, la detección temprana de errores y la mejora continua del producto.

Scrum divide el proyecto en iteraciones cortas llamadas sprints, al final de las cuales se obtiene una parte funcional del proyecto. Este enfoque resulta adecuado para proyectos de desarrollo, ya que permite avanzar de forma progresiva y flexible, adaptándose a posibles cambios en los requisitos.

5. Diseño de solución

5.1. Tecnologías utilizadas

Para el desarrollo de la aplicación se emplearán distintas tecnologías, seleccionadas en función de su eficiencia, compatibilidad y adecuación a los objetivos del proyecto. Estas tecnologías permiten desarrollar una aplicación multiplataforma, escalable y con posibilidad de monetización mediante publicidad.

5.1.1 Ionic

Ionic es un framework de desarrollo híbrido que permite crear aplicaciones móviles multiplataforma utilizando tecnologías web. Su uso facilita el desarrollo de una única base de código que puede ejecutarse tanto en dispositivos Android como iOS.

Ionic proporciona componentes visuales optimizados para dispositivos móviles, lo que mejora la experiencia de usuario y agiliza el proceso de desarrollo.

5.1.2 React

React es una biblioteca de JavaScript orientada a la creación de interfaces de usuario basadas en componentes. Permite desarrollar interfaces dinámicas, reutilizables y eficientes.

La combinación de React con Ionic facilita la creación de una aplicación modular y organizada, mejorando el mantenimiento y la escalabilidad del proyecto.

5.1.3 TypeScript

TypeScript es un superset de JavaScript que añade tipado estático al lenguaje. Su uso permite detectar errores durante el desarrollo, mejorar la legibilidad del código y facilitar el mantenimiento del proyecto.

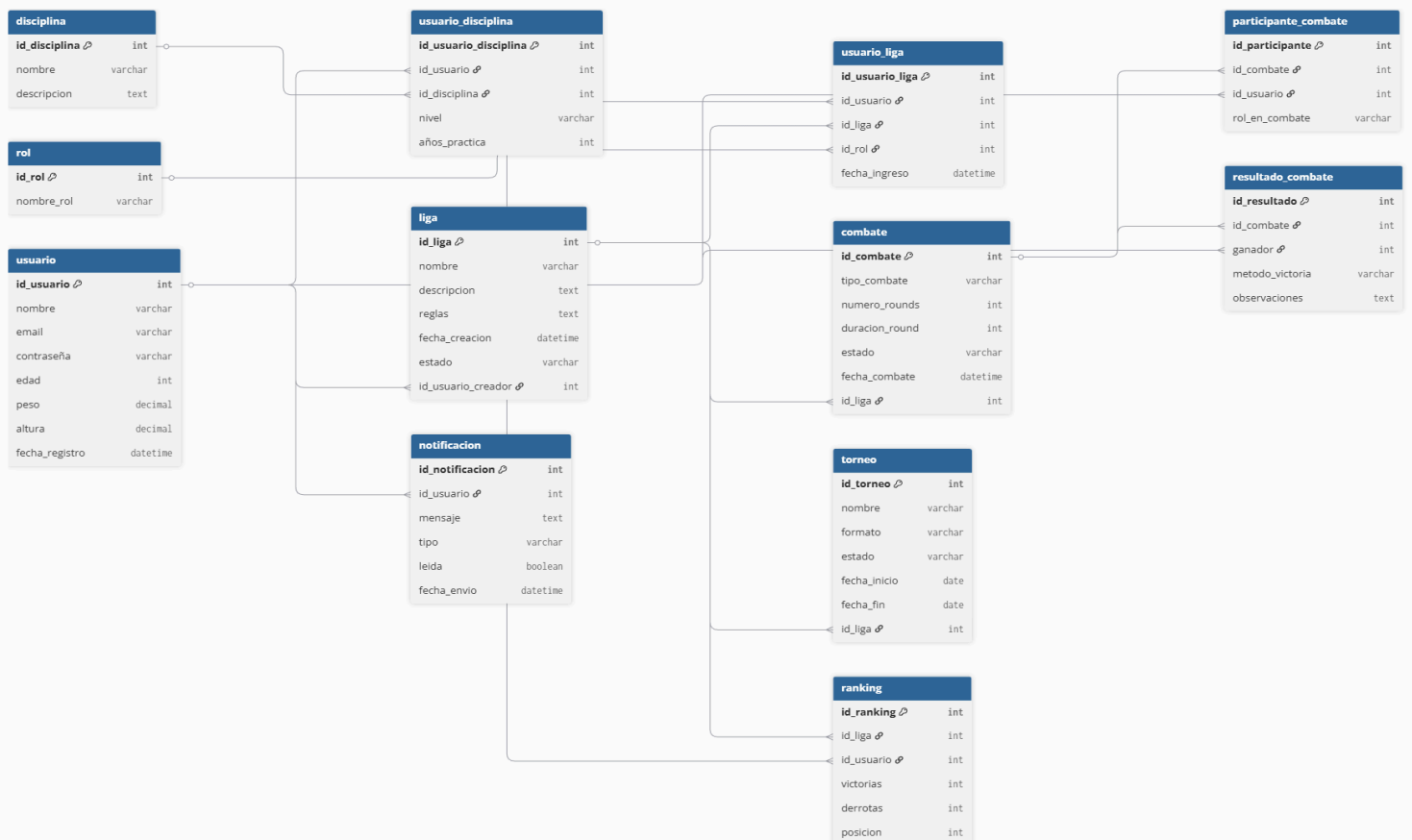
Gracias a TypeScript, el desarrollo de la aplicación resulta más seguro y estructurado, especialmente en proyectos de mayor complejidad.

5.1.4 MariaDB

MariaDB es un sistema gestor de bases de datos relacional, derivado de MySQL, que destaca por su rendimiento, estabilidad y compatibilidad.

Se utilizará para almacenar y gestionar la información necesaria de la aplicación, garantizando la integridad de los datos y un acceso eficiente a los mismos.

En el siguiente diagrama se especifican las tablas de la BBDD:



5.1.5 Google AdMob

Google AdMob es una plataforma de monetización que permite integrar anuncios publicitarios dentro de aplicaciones móviles. En este proyecto se utilizará AdMob para mostrar banners de publicidad, generando ingresos a partir del uso de la aplicación.

La integración de AdMob se realizará de forma no intrusiva, asegurando que la publicidad no afecte negativamente a la experiencia de usuario.

5.1.6 Backend (Node.js y Express)

El backend de la aplicación se desarrollará utilizando Node.js junto con el framework Express, permitiendo la creación de una API REST robusta y escalable.

Node.js proporciona un entorno de ejecución eficiente y orientado a eventos, ideal para aplicaciones que requieren manejar múltiples solicitudes simultáneas, como es el caso de FightClubX. Por su parte, Express facilita la creación de rutas, la gestión de peticiones HTTP y la implementación de la lógica de negocio.

El backend será el encargado de:

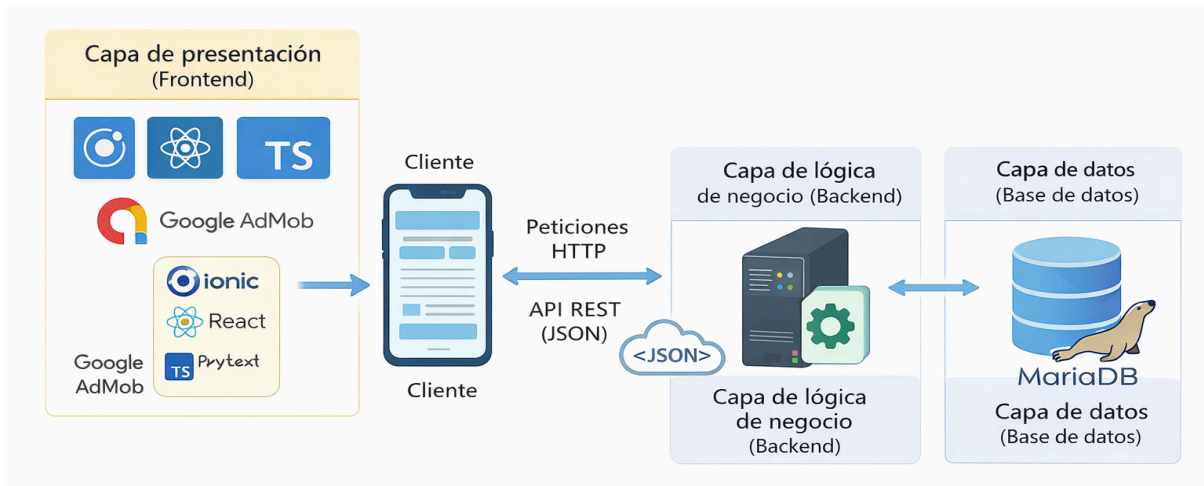
- Gestionar la autenticación y autorización de usuarios.
- Controlar la lógica de creación de combates, torneos y ligas.
- Gestionar los rankings y resultados.
- Enviar notificaciones a los usuarios.
- Comunicar el frontend con la base de datos MariaDB.

5.1.6 Otras herramientas

Además de las tecnologías principales, se utilizarán herramientas complementarias como:

- Visual Studio Code como entorno de desarrollo.
- Git para el control de versiones.
- Navegador web y emuladores para pruebas y depuración.

En el siguiente diagrama podemos observar con claridad cuál será la arquitectura de nuestro sistema:



5.2. Objetivos esperados del diseño

5.2.1. Interfaz de usuario

Se espera realizar un interfaz de usuario cómoda para el usuario y que implemente todos los campos de la app siguiendo la gama estética del siguiente mockUp de la página principal.



5.2.2. Diseño del Backend

El diseño del backend tiene como objetivo principal proporcionar una base sólida, segura y escalable que soporte todas las funcionalidades de la aplicación.

Se espera que el backend:

- Centralice toda la lógica de negocio del sistema.
- Valide las operaciones realizadas por los usuarios.
- Garantice la integridad y consistencia de los datos almacenados.
- Permita una comunicación eficiente con el frontend mediante una API REST.

La estructura del backend se diseñará de forma modular, separando claramente:

- Controladores (gestión de peticiones).
- Servicios (lógica de negocio).
- Modelos (acceso a datos).
- Rutas (endpoints de la API).

Este enfoque facilitará futuras ampliaciones del sistema, como la incorporación de nuevas funcionalidades o mejoras en el rendimiento.

5.2.3. Generación de combates desde el backend

Uno de los objetivos clave del diseño del backend es la generación automática de combates.

El backend será responsable de:

- Emparejar a los participantes según el tipo de evento (combate rápido, torneo o liga).
- Tener en cuenta criterios como ranking, rol y disponibilidad.
- Registrar los combates en la base de datos.
- Actualizar resultados y rankings una vez finalizado el combate.

Este proceso se realizará de forma automática, reduciendo la intervención manual y asegurando un funcionamiento justo y eficiente del sistema.

5.2.4. Gestión de notificaciones desde el backend

El backend tendrá como objetivo gestionar la generación y el envío de notificaciones a los usuarios cuando se produzcan eventos relevantes dentro de la aplicación.

Entre estos eventos se incluyen:

- Creación o inicio de combates.
- Inscripción en torneos.
- Cambios en ligas o rankings.
- Apertura de votaciones o encuestas.

El sistema de notificaciones permitirá mejorar la experiencia de usuario, manteniéndolo informado en tiempo real sobre la actividad de la aplicación. Las notificaciones se enviarán al dispositivo móvil del usuario como una notificación push de la app.

6. Ejecución

6.1. Alcance de 1ª fase de ejecución

En la fase 1 del proyecto se tratará todos los requisitos funcionales tratados en la documentación del proyecto

6.2. Planificación

El proyecto se dividirá en 8 sprints:

Sprint 1: 17 febrero – 24 febrero 2026

- Crear la base de datos
- Diseñar la interfaz básica del proyecto
- Prototipo de Generación de combates

Sprint 2: 24 febrero – 10 marzo 2026

-Crear Backend de la App (Excluyendo la autenticación de usuarios)

- Crear estructura básica del backend

- Crear config del back

- Crear controller de combates

- Crear service de combates

-Implementar el Backend en la interfaz

- Implementar backend de combates a la interfaz

Sprint 3: 11 marzo – 21 marzo 2026

-Implementar sistema de autenticación de usuarios

-Añadir calendario de eventos a la interfaz de usuario

-Implementar al completo la Generación de combates

Sprint 4: 22 marzo – 1 abril 2026

-Implementar sistemas de Torneos

- Inscripción de participantes

- Generación de bracket

- Avance de rondas

- Finalizar torneo

- Frontend

-Implementar Ligas simplificadas

- Conectar back simplificado al front

Sprint 5: 2 abril – 12 abril 2026

- Completar el sistema Ligas junto al de Roles de la Liga
- Implementar sistema de historial de combates

Sprint 6: 13 abril – 23 abril 2026

- Crear sistema de notificaciones
- Implementar Votaciones y encuestas a las ligas

Sprint 7: 24 abril – 4 mayo 2026

- Añadir Admob al proyecto a través de su API
- Crear sistema de Rankings
- Realizar pruebas

Sprint 8: 5 mayo – 15 mayo 2026

- Corrección de errores
- Exportación de la app a un dispositivo móvil para volver a realizar pruebas en él

6.3. Evolución y seguimiento

6.3.1. Reunión 24/03/2026

Primera entrega del documento con sprint 1 realizado y planificación del resto de sprints.

6.3.2. Reunión 10/03/2026

Retraso en el sprint 2 ,falta por terminar el service de combates y su respectiva implementación en la interfaz

6.3.3. Reunión 18/03/2026

Completado el sprint 2,se ha iniciado el sprint 3 con los siguientes puntos

- Implementar sistema de autenticación de usuarios realizado
- Falta completar la implementación de combates al completo y realizar el sistema de calendario en la interfaz

6.3.4. Reunión 24/03/2026

Sprint 3 finalizado:

- Pestaña de calendario implementado a la interfaz(Sin lógica aún)
- Generación de combates implementada

Sprint 4 iniciado

6.3.5. Reunión 07/04/2026

Sprint 4 finalizado al completo.

Sprint 5 iniciado

6.3.6. Reunión 14/04/2026

Sprint 5 finalizado

6.3.7. Reunión 4/05/2026

Sprint 6 en desarrollo, tras observaciones del sprint 7 sobre adMob se ha ampliado el tiempo del Sprint 6 y reorganizado el Sprint 7 , con la reorganización quedaría de la siguiente manera el calendario de Sprints:

Sprint 6: 13 abril – 6 abril 2026

-Crear sistema de notificaciones

-Implementar Votaciones y encuestas a las ligas

Sprint 7: 6 abril – 15 mayo 2026

-Crear sistema de Rankings

-Realizar pruebas

-Corrección de errores

-Exportación de la app a un dispositivo móvil para volver a realizar pruebas en él

7. Pruebas y despliegue

7.1. Estrategia de pruebas

1. PRUEBAS DE BACKEND

A. Pruebas Unitarias (Servicios):

- Verificar que la lógica de cálculo del Ranking sea correcta (bonos por experiencia, puntos base).
- Validar que el servicio de autenticación cifre correctamente las contraseñas.
- Comprobar que el TournamentService gestione correctamente los estados de los torneos.

B. Pruebas de Integración (API):

- Probar el flujo completo de registro y login de usuario (/api/auth).
- Simular la creación de un combate y el envío de un resultado para verificar que se actualicen las tablas de resultados y ranking.
- Verificar que los endpoints de eventos devuelvan la lista correcta de eventos activos.

2. PRUEBAS DE FRONTEND

A. Pruebas de Componentes:

- Verificar que el formulario de Login muestre errores cuando los campos están vacíos.
- Comprobar que la página de Ranking renderice correctamente la lista de luchadores obtenida del servicio.
- Validar que los botones de navegación redirijan a las rutas adecuadas.

B. Pruebas de Servicios (Mocks):

- Simular respuestas de la API para probar cómo el frontend maneja estados de error (ej: servidor caído, 401 Unauthorized).
- Verificar que los datos recibidos de la API se transformen correctamente al modelo de datos del frontend.

C. Pruebas de Interfaz de Usuario (UI/UX):

- Comprobar la responsividad en diferentes tamaños de pantalla (móvil, tablet).

3. PRUEBAS END-TO-END

- Flujo de combate: Árbitro inicia combate -> Registra resultados de los rounds -> Finaliza combate -> Se verifica actualización instantánea en el Ranking.

7.2. Estrategia de despliegue

La aplicación se terminara de desplegar la semana antes a la presentación del proyecto siguiendo la siguiente estrategia:

1. INFRAESTRUCTURA DE DATOS (Aiven)

Se utilizará Aiven como proveedor de base de datos por su alta fiabilidad y compatibilidad nativa con MariaDB.

- Acción: Crear una instancia de MariaDB en el plan Free Tier.
- Configuración: Se obtendrán las credenciales (Host, Puerto, Usuario, Contraseña) y se integrarán en el entorno de producción.
- Ejecución: Se cargará el archivo 'schema.sql' para inicializar la estructura de tablas.

2. ALOJAMIENTO DEL BACKEND (Render)

Se desplegará la API de Node.js en Render debido a su facilidad de integración con repositorios de GitHub.

- Acción: Conectar el repositorio 'Proyecto-Multimedia' a un nuevo Web Service en Render.
- Build: Se ejecutará 'npm install' y 'npm run build' en la carpeta Backend/Back-FightClubX.
- Runtime: El servidor se iniciará desde la carpeta 'dist/' generada por TypeScript.
- Variables: Se configurarán JWT_SECRET y los datos de conexión de Aiven en el panel de Environment.

3. ALOJAMIENTO DEL FRONTEND (Vercel)

Se utilizará Vercel para el frontend Angular por su optimización específica para este framework y su red de distribución (CDN) global.

- Acción: Importar el proyecto en Vercel apuntando al subdirectorio FightClubX/FightClubX.
- API Connect: Se configurará la variable de entorno API_URL para que el frontend se comunique con la URL asignada por Render.
- Despliegue: Se realizará la compilación de producción mediante 'ng build' y se generará la URL pública final (ej: fightclubx.vercel.app).

4. PROTOCOLO DE CONEXIÓN Y SEGURIDAD

- Se forzará el uso de HTTPS en todas las comunicaciones.
- Se implementará CORS en el Backend para permitir peticiones únicamente desde el dominio de Vercel.
- Se verificará que el 'Health Check' de la API responda correctamente antes de considerar el despliegue como finalizado.

8. Conclusiones y propuestas de mejoras

8.1. Conclusiones

-El proyecto se ha podido llevar a cabo casi al completo ,quedando excluidos dos puntos : las pruebas en dispositivo móvil y la implementación de AdMob

-El proyecto ha sido muy útil para practicar a aprender a trabajar con agentes de IA por consola de comandos y ha aportado mucha experiencia de usa de estas, además de facilitar el desarrollo del proyecto

-Un error que cometí a la hora de plantear el proyecto fue informarme erróneamente sobre AdMob ,lo cual ha impedido su implementación en el proyecto final

8.2. Propuestas de mejora

-La principal mejora que veo es implementar adecuadamente AdMob

- A la hora de realizar el seguimiento utilizar Jira en lugar de Trello ,ya que este no abarca opciones de seguimiento extensas y está demasiado simplificado

-Por último sería interesante utilizar skills y plugins para Gemini CLI a la hora de generar la interfaz de usuario y así obtener un mejor resultado

9. Bibliografía

- Ionic Team. *Ionic Framework Documentation*. <https://ionicframework.com/docs>
- React Team. *React – A JavaScript library for building user interfaces*. <https://react.dev>
- Microsoft.. *TypeScript Documentation*. <https://www.typescriptlang.org/docs/>
- MariaDB Foundation. *MariaDB Documentation*. <https://mariadb.org/documentation/>
- Google. *Google AdMob – Monetize your app*. <https://developers.google.com/admob>
- dbdiagram.io. *DBDiagram – Database diagram tool*. <https://dbdiagram.io>
- Git SCM. *Git Documentation*. <https://git-scm.com/doc>
- Atlassian. *Jira Software Documentation*. <https://www.atlassian.com/software/jira>